Application & Data Security (CS4021) — Assignment 1

Threat Modeling, Vulnerability Exploitation, and Mitigation (OWASP Top 10)

Student Name: Aditya Ranjan Kumar

Roll Number: CS-2341901

Class & Section: 4CSE9

Course Instructor: Dr. Ajay Prasad

Submission Date: 15-10-2026

GitHub Repository: https://github.com/Adityaaop/Ads

1. INTRODUCTION & SELECTED OWASP CATEGORY

For this assignment, I selected **OWASP Top 10 (2021) Category A01:2021 – Broken Access Control**.

Broken Access Control currently holds the **#1 position** in the OWASP Top 10. Access control mechanisms enforce authorization policies so users cannot access resources or execute actions outside their intended permissions. When access controls are broken, attackers can view unauthorized files, steal server configurations, or access sensitive user records.

In this practical lab, I designed and implemented a dummy web application demonstrating **Path Traversal (CWE-22)** and **Improper Access Control (CWE-284)** in a file download feature, and also applied **Insecure Direct Object Reference (CWE-639)** mitigations.

2. MAPPING OF CWES TO OWASP A01:2021

OWASP associates multiple Common Weakness Enumerations (CWEs) with Broken Access Control. Below is my mapping of these weaknesses, highlighting the specific ones demonstrated and fixed in this project:

CWE ID	Weakness Name	Severity	Role in My Assignment
CWE-22	**Path Traversal (Improper Limitation of Pathname)**	High	**Primary Vulnerability Exploit**
CWE-284	**Improper Access Control**	High	**Core Architectural Flaw**
CWE-639	**Authorization Bypass via User Key (IDOR)**	High	**Remediated via Indirect Tokens**
CWE-552	Files Accessible to External Parties	Medium	Demonstrated via .env leak
CWE-200	Exposure of Sensitive Information	Medium	Mitigated via safe 403 responses

Additional CWEs mapped under OWASP A01:2021 include CWE-23, CWE-73, CWE-276, CWE-706, CWE-862, CWE-863, and CWE-918.

3. DUMMY APPLICATION SETUP & VULNERABILITY ANALYSIS

I created an academic document distribution portal in Python (Flask). The app allows students to download course materials from `storage/public_docs/`, while confidential admin credentials reside in `storage/confidential/`.

Vulnerable Code in `vulnerable_app/app.py`:

```
@app.route("/download")
def download_vulnerable():
    filename = request.args.get("file")
    # Insecure direct join without validation:
    file_path = os.path.join(PUBLIC_DOCS_DIR, filename)
    if not os.path.exists(file_path):
        return f"File not found: {filename}", 404
    return send_file(file_path)
```

Vulnerability Mechanism:

1. `os.path.join()` in Python does not sanitize relative path sequences (`../`).
2. Passing `file=../confidential/admin_db_credentials.env` escapes the public directory and reads internal database credentials.
3. Passing `file=../../.env` navigates two levels up to the server root, exposing master secret keys and database connection URIs.

Failure of Naive Blacklisting (`.replace('../', '')`):

A naive mitigation attempted by developers is stripping `../` with string replacement. In `vulnerable_app/app.py`, I implemented `/download_naive`:

```
filtered_filename = raw_filename.replace("../", "")
```

Because single-pass replacement is non-recursive, sending a nested sequence like `....//` causes the inner `../` to be stripped, leaving behind `../`: `....//confidential/admin_db_credentials.env` → `../confidential/admin_db_credentials.env`. The attack succeeds completely with **HTTP 200 OK**.

4. THREAT MODELING (STRIDE)

I evaluated the document download feature against the six STRIDE threat categories:

STRIDE Category	Threat in My Application	Component	Potential Impact	Mitigation Applied
Spoofing	Attacker modifies document IDs to access another student's file.	Query parameter (`doc_id`)	Unauthorized document download.	Server-side dictionary mapping (`AUTHORIZED_DOCUMENTS`).
Tampering	Attacker injects `../` or `....//` into filename parameter.	`/download?file=...`	Filesystem path redirection to restricted directories.	Regex allowlisting & canonical boundary validation (`os.path.realpath`).
Repudiation	Attacker downloads sensitive files without leaving an audit trail.	Access Logging	Inability to detect or trace data exfiltration.	Structured logging of IP address, document token, and HTTP status.
Information Disclosure	Path traversal reads `.env` and database credentials.	Local File System	Confidentiality loss; database credentials stolen.	Canonical containment check via `os.path.commonpath`.
Denial of Service	Attacker requests infinite device streams (e.g., `/dev/urandom`).	OS File Descriptors	Worker process hangs; memory exhaustion.	Enforce `os.path.isfile()` check and size constraints.
Elevation of Privilege	Attacker extracts `SECRET_KEY` to forge admin JWT/session cookies.	Session Management	Administrative takeover of portal.	Store secrets outside web root; block traversal; enforce least privilege.

5. RISK ASSESSMENT (DREAD MODEL)

I calculated risk scores using the standard DREAD formula: **Risk Score = (Damage + Reproducibility + Exploitability + Affected Users + Discoverability) / 5**

DREAD Dimension	Pre-Fix	Pre-Mitigation Rationale	Post-Fix	Post-Mitigation Rationale
Damage Potential (D)	**9 / 10**	Leaking `.env` reveals database passwords and crypto keys.	**1 / 10**	Access rejected with 403; zero sensitive bytes disclosed.
Reproducibility (R)	**10 / 10**	100% reproducible on every GET request with `../`.	**1 / 10**	Canonical check reliably blocks the traversal every time.
Exploitability (E)	**10 / 10**	Trivial; can be executed from any standard browser URL bar.	**1 / 10**	Constrained to server-defined tokens; cannot be bypassed.
Affected Users (A)	**8 / 10**	Leaked database credentials compromise all student records.	**0 / 10**	Zero users affected as vulnerability is completely sealed.
Discoverability (D)	**9 / 10**	Security scanners (ZAP, Burp) flag path traversal immediately.	**2 / 10**	Server returns generic 403 errors with no path information.
Overall DREAD Average	9.2 / 10 (CRITICAL)	**Immediate remediation required**	1.0 / 10 (NEGLIGIBLE)	**Residual risk is fully acceptable**

6. THREAT AND EXPLOIT DEMONSTRATION

I executed proof-of-concept attacks against the vulnerable application using `scripts/exploit_demo.py` and `curl`:

1. **Normal Access:** `GET /download?file=syllabus_cs2341901.txt` → **HTTP 200 OK** (Returns syllabus).
2. **Stealing Database Credentials:** `GET /download?file=../confidential/admin_db_credentials.env` → **HTTP 200 OK**

   ```
   DB_HOST=10.0.12.44
   DB_USER=ads_admin
   DB_PASS=SuperSecretP@ssw0rd2026!
   JWT_SECRET_KEY=9f83ab2018cf41a980e927164b38d0e7
   ```

3. **Stealing Root Server Secrets:** `GET /download?file=../../.env` → **HTTP 200 OK** (Exfiltrates production DB connection strings and AWS API keys).
4. **Bypassing Naive Filter:** `GET /download_naive?file=....//confidential/admin_db_credentials.env` → **HTTP 200 OK** (Bypasses single-pass replacement).

7. MITIGATION IMPLEMENTATION & CODE COMPARISON

In `secure_app/app.py`, I implemented a Defense-in-Depth mitigation strategy:

- **Canonical Path Resolution:** Using `os.path.realpath()` and `os.path.commonpath()` to ensure target files remain inside `public_docs`.
- **Indirect Object References (IDOR Prevention):** Using token IDs (`doc_id=doc_01`) so users never directly specify filesystem paths.
- **Regex Allowlisting:** Rejecting illegal characters, directory separators, and traversal markers.
- **Information Leakage Prevention:** Returning generic `403 Forbidden` responses without revealing directory structure.

Side-by-Side Code Comparison:

```
--- vulnerable_app/app.py (Vulnerable)
+++ secure_app/app.py (Hardened)
@@ -18,14 +18,34 @@
-@app.route("/download")
-def download_vulnerable():
-    filename = request.args.get("file")
-    # INSECURE: Direct concatenation allows ../ traversal
-    file_path = os.path.join(PUBLIC_DOCS_DIR, filename)
-    if not os.path.exists(file_path):
-        return f"File not found: {filename}", 404
-    return send_file(file_path)
+@app.route("/download_secure")
+def download_secure():
+    doc_id = request.args.get("doc_id")
+    # DEFENSE 1: Indirect Object Reference mapping
+    if doc_id not in AUTHORIZED_DOCUMENTS:
+        return "Access Denied: Invalid document identifier.", 403
+
+    safe_name = AUTHORIZED_DOCUMENTS[doc_id]["filename"]
+    full_path = os.path.join(PUBLIC_DOCS_DIR, safe_name)
+
+    # DEFENSE 2: Canonical boundary verification
+    if not is_safe_path(PUBLIC_DOCS_DIR, full_path):
+        return "Access Denied: Path escapes boundary.", 403
+
+    return send_file(full_path)
```

8. AUTOMATED TEST SUITE VERIFICATION

I wrote an automated `unittest` suite in `scripts/test_suite.py` containing 12 test assertions:

```
======================================================================
RUNNING AUTOMATED SECURITY TEST SUITE (CS4021 - Assignment-1)
Student: Aditya Ranjan Kumar | CS-2341901 | 4CSE9
Instructor: Dr. Ajay Prasad
======================================================================

test_01_authorized_token_download (__main__.TestSecureHardenedApplication) ... ok
test_02_tampered_doc_id_traversal (__main__.TestSecureHardenedApplication) ... ok
test_03_tampered_doc_id_nonexistent (__main__.TestSecureHardenedApplication) ... ok
test_04_direct_name_path_traversal_blocked (__main__.TestSecureHardenedApplication) ... ok
test_05_direct_name_nested_traversal_blocked (__main__.TestSecureHardenedApplication) ... ok
test_06_direct_name_special_characters_blocked (__main__.TestSecureHardenedApplication) ... ok
test_07_direct_name_legitimate_download (__main__.TestSecureHardenedApplication) ... ok
test_01_legitimate_download (__main__.TestVulnerableApplication) ... ok
test_02_path_traversal_confidential_env (__main__.TestVulnerableApplication) ... ok
test_03_path_traversal_root_env (__main__.TestVulnerableApplication) ... ok
test_04_naive_filter_bypass (__main__.TestVulnerableApplication) ... ok
test_05_missing_parameter (__main__.TestVulnerableApplication) ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.014s
OK
```

9. CONCLUSION

This lab demonstrates why **Broken Access Control** is ranked #1 in the OWASP Top 10. Direct file path concatenation without canonical path checks allows attackers to read sensitive files with simple `../` payloads. Naive string replacement like `.replace('../', '')` is ineffective against nested sequences like `....//`. The best and most reliable defenses are **canonical path validation** and **indirect object references**.